

DEEPSKILLING – JAVA FSE REACT
Algorithms and Data Structures Exercises

Name: DARSHAN DONTIBOINA

SUPERSET ID: 8001653

Question – 1

2,3.

Code:

```
import java.util.*;

class Product {
    int productId;
    String productName;
    int quantity;
    double price;

    public Product(int productId, String productName,
        int quantity, double price) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
    }

    @Override
    public String toString() {
        return productId + " " + productName +
            " Qty:" + quantity + " Price:" + price;
    }
}

public class Inventory {
    HashMap<Integer, Product> products = new HashMap<>();

    public void addProduct(Product p) {
        products.put(p.productId, p);
    }

    public void updateProduct(int id, int quantity, double price) {
        Product p = products.get(id);

        if (p != null) {
            p.quantity = quantity;
            p.price = price;
        }
    }

    public void deleteProduct(int id) {
        products.remove(id);
    }

    public void displayProducts() {
        products.values().forEach(System.out::println);
    }

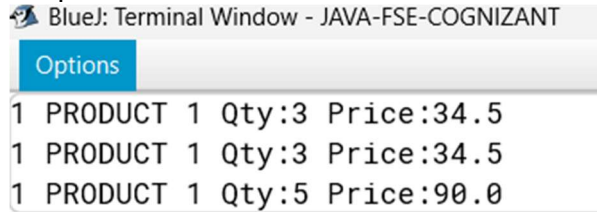
    public static void main(String args[]){
        Scanner in=new Scanner(System.in);
        Product P=new Product(1,"PRODUCT 1", 3, 34.50);
        System.out.println(P.toString());
    }
}
```

```

        Inventory i=new Inventory();
        i.addProduct(P);
        i.displayProducts();
        i.updateProduct(1,5,90.00);
        i.displayProducts();
        i.deleteProduct(1);
        i.displayProducts();
    }
}

```

Output:



```

BlueJ: Terminal Window - JAVA-FSE-COGNIZANT
Options
1 PRODUCT 1 Qty:3 Price:34.5
1 PRODUCT 1 Qty:3 Price:34.5
1 PRODUCT 1 Qty:5 Price:90.0

```

Question – 2

2,3.

Code:

```

import java.util.Arrays;
class Product {
    int productId;
    String productName;
    String category;
    public Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    @Override
    public String toString() {
        return "Product ID: " + productId +
            ", Name: " + productName +
            ", Category: " + category;
    }
}

public class Question2 {
    public static Product linearSearch(Product[] products, String name) {
        for (Product p : products) {
            if (p.productName.equalsIgnoreCase(name))
                return p;
        }
        return null;
    }

    public static Product binarySearch(Product[] products, String name) {
        int left = 0;
        int right = products.length - 1;
        while (left <= right) {
            int mid = (left + right) / 2;
            int compare = products[mid].productName.compareToIgnoreCase(name);

            if (compare == 0)

```

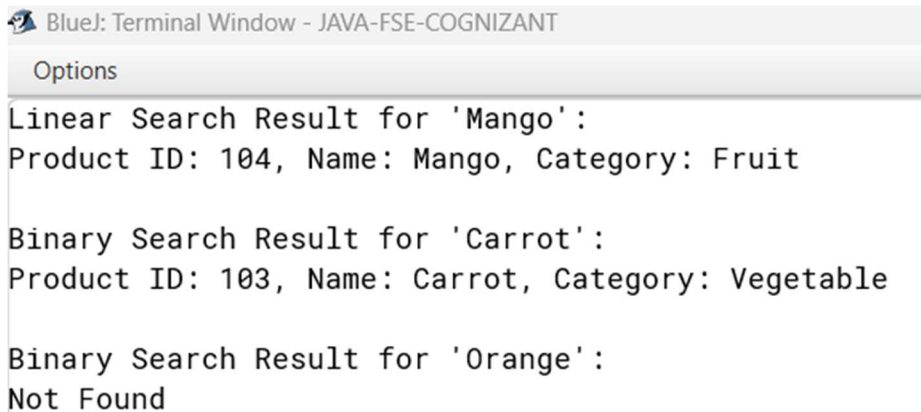
```

        return products[mid];
    if (compare < 0)
        left = mid + 1;
    else
        right = mid - 1;
    }
    return null;
}

public static void main(String[] args) {
    Product[] products = {
        new Product(101, "Apple", "Fruit"),
        new Product(102, "Banana", "Fruit"),
        new Product(103, "Carrot", "Vegetable"),
        new Product(104, "Mango", "Fruit"),
        new Product(105, "Potato", "Vegetable")
    };
    Arrays.sort(products, (a, b) ->
a.productName.compareToIgnoreCase(b.productName));
    String searchName1 = "Mango";
    Product result1 = linearSearch(products, searchName1);
    System.out.println("Linear Search Result for " + searchName1 + " :");
    System.out.println(result1 != null ? result1 : "Not Found");
    String searchName2 = "Carrot";
    Product result2 = binarySearch(products, searchName2);
    System.out.println("\nBinary Search Result for " + searchName2 + " :");
    System.out.println(result2 != null ? result2 : "Not Found");
    String searchName3 = "Orange";
    Product result3 = binarySearch(products, searchName3);
    System.out.println("\nBinary Search Result for " + searchName3 + " :");
    System.out.println(result3 != null ? result3 : "Not Found");
}
}

```

Output:



```

BlueJ: Terminal Window - JAVA-FSE-COGNIZANT
Options
Linear Search Result for 'Mango':
Product ID: 104, Name: Mango, Category: Fruit

Binary Search Result for 'Carrot':
Product ID: 103, Name: Carrot, Category: Vegetable

Binary Search Result for 'Orange':
Not Found

```

Question – 3

2,3.

Code:

```

class Order {
    int orderId;
    String customerName;
}

```

```

double totalPrice;

public Order(int orderId, String customerName, double totalPrice) {
    this.orderId = orderId;
    this.customerName = customerName;
    this.totalPrice = totalPrice;
}

@Override
public String toString() {
    return "Order{" +
        "orderId=" + orderId +
        ", customerName=" + customerName + "\" +
        ", totalPrice=" + totalPrice +
        "\"";
}

public static void bubbleSort(Order[] orders) {
    int n = orders.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (orders[j].totalPrice > orders[j + 1].totalPrice) {
                Order temp = orders[j];
                orders[j] = orders[j + 1];
                orders[j + 1] = temp;
            }
        }
    }
}

public static void quickSort(Order[] orders, int low, int high) {
    if (low < high) {
        int pi = partition(orders, low, high);
        quickSort(orders, low, pi - 1);
        quickSort(orders, pi + 1, high);
    }
}

private static int partition(Order[] orders, int low, int high) {
    double pivot = orders[high].totalPrice;
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (orders[j].totalPrice <= pivot) {
            i++;
            Order temp = orders[i];
            orders[i] = orders[j];
            orders[j] = temp;
        }
    }
    Order temp = orders[i + 1];
    orders[i + 1] = orders[high];
    orders[high] = temp;
    return i + 1;
}

public static void main(String[] args) {

```

```

Order[] orders = {
    new Order(101, "Amit", 2500.50),
    new Order(102, "Bhavana", 1500.75),
    new Order(103, "Chandan", 3200.00),
    new Order(104, "Deepa", 1800.25),
    new Order(105, "Eshan", 2900.10)
};

System.out.println("Original Orders:");
for (Order o : orders) {
    System.out.println(o);
}

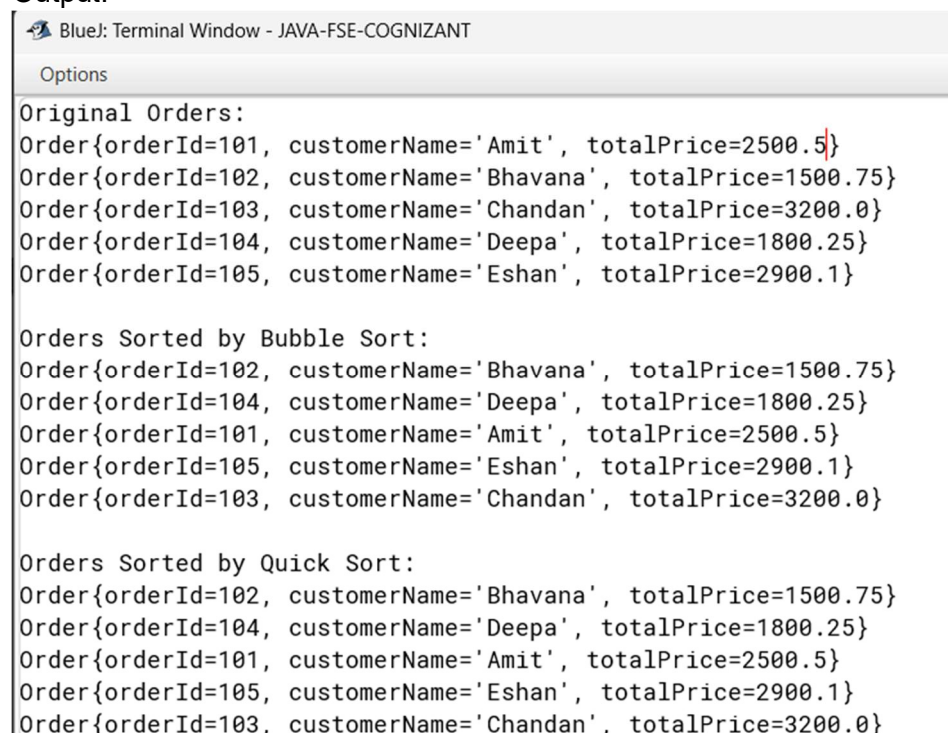
bubbleSort(orders);
System.out.println("\nOrders Sorted by Bubble Sort:");
for (Order o : orders) {
    System.out.println(o);
}

Order[] orders2 = {
    new Order(101, "Amit", 2500.50),
    new Order(102, "Bhavana", 1500.75),
    new Order(103, "Chandan", 3200.00),
    new Order(104, "Deepa", 1800.25),
    new Order(105, "Eshan", 2900.10)
};

quickSort(orders2, 0, orders2.length - 1);
System.out.println("\nOrders Sorted by Quick Sort:");
for (Order o : orders2) {
    System.out.println(o);
}
}
}

```

Output:



```

Blue: Terminal Window - JAVA-FSE-COGNIZANT
Options
Original Orders:
Order{orderId=101, customerName='Amit', totalPrice=2500.5}
Order{orderId=102, customerName='Bhavana', totalPrice=1500.75}
Order{orderId=103, customerName='Chandan', totalPrice=3200.0}
Order{orderId=104, customerName='Deepa', totalPrice=1800.25}
Order{orderId=105, customerName='Eshan', totalPrice=2900.1}

Orders Sorted by Bubble Sort:
Order{orderId=102, customerName='Bhavana', totalPrice=1500.75}
Order{orderId=104, customerName='Deepa', totalPrice=1800.25}
Order{orderId=101, customerName='Amit', totalPrice=2500.5}
Order{orderId=105, customerName='Eshan', totalPrice=2900.1}
Order{orderId=103, customerName='Chandan', totalPrice=3200.0}

Orders Sorted by Quick Sort:
Order{orderId=102, customerName='Bhavana', totalPrice=1500.75}
Order{orderId=104, customerName='Deepa', totalPrice=1800.25}
Order{orderId=101, customerName='Amit', totalPrice=2500.5}
Order{orderId=105, customerName='Eshan', totalPrice=2900.1}
Order{orderId=103, customerName='Chandan', totalPrice=3200.0}

```

Question – 4

2,3.

Code:

```
class Employee {
    private int employeeId;
    private String name;
    private String position;
    private double salary;

    public Employee(int employeeId, String name, String position, double salary) {
        this.employeeId = employeeId;
        this.name = name;
        this.position = position;
        this.salary = salary;
    }

    public int getEmployeeId() { return employeeId; }
    public void setEmployeeId(int employeeId) { this.employeeId = employeeId; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getPosition() { return position; }
    public void setPosition(String position) { this.position = position; }

    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }

    @Override
    public String toString() {
        return "ID: " + employeeId + " | Name: " + name + " | Position: " + position + " | Salary: " + salary;
    }
}

class EmployeeManagement {
    private Employee[] employees;
    private int size;

    public EmployeeManagement(int capacity) {
        employees = new Employee[capacity];
        size = 0;
    }

    public boolean addEmployee(Employee emp) {
        if (size >= employees.length) {
            System.out.println("Error: Cannot add employee. Array is full!");
            return false;
        }
        employees[size] = emp;
        size++;
        System.out.println("Employee added successfully: " + emp.getName());
        return true;
    }

    public Employee searchEmployee(int employeeId) {
        for (int i = 0; i < size; i++) {
            if (employees[i].getEmployeeId() == employeeId) {
                return employees[i];
            }
        }
    }
}
```

```

    }
}
return null;
}
public void traverseEmployees() {
    if (size == 0) {
        System.out.println("No employee records found.");
        return;
    }
    System.out.println("\n--- Current Employee Records ---");
    for (int i = 0; i < size; i++) {
        System.out.println(employees[i]);
    }
    System.out.println("-----");
}
public boolean deleteEmployee(int employeeId) {
    int indexToDelete = -1;

    for (int i = 0; i < size; i++) {
        if (employees[i].getEmployeeId() == employeeId) {
            indexToDelete = i;
            break;
        }
    }
    if (indexToDelete == -1) {
        System.out.println("Error: Employee with ID " + employeeId + " not found.");
        return false;
    }

    for (int i = indexToDelete; i < size - 1; i++) {
        employees[i] = employees[i + 1];
    }

    employees[size - 1] = null;
    size--;
    System.out.println("Employee with ID " + employeeId + " deleted successfully.");
    return true;
}
}
public class question4 {
    public static void main(String[] args) {
        EmployeeManagement manager = new EmployeeManagement(5);

        System.out.println("--- Testing Add Operations ---");
        manager.addEmployee(new Employee(101, "Alice Smith", "Software Engineer",
85000));
        manager.addEmployee(new Employee(102, "Bob Jones", "Product Manager", 95000));
        manager.addEmployee(new Employee(103, "Charlie Brown", "UX Designer", 75000));
        manager.traverseEmployees();
        System.out.println("\n--- Testing Search Operation ---");
        int targetId = 102;
        Employee found = manager.searchEmployee(targetId);
        if (found != null) {
            System.out.println("Found Record: " + found);
        } else {
            System.out.println("Employee with ID " + targetId + " not found.");
        }
    }
}

```

```

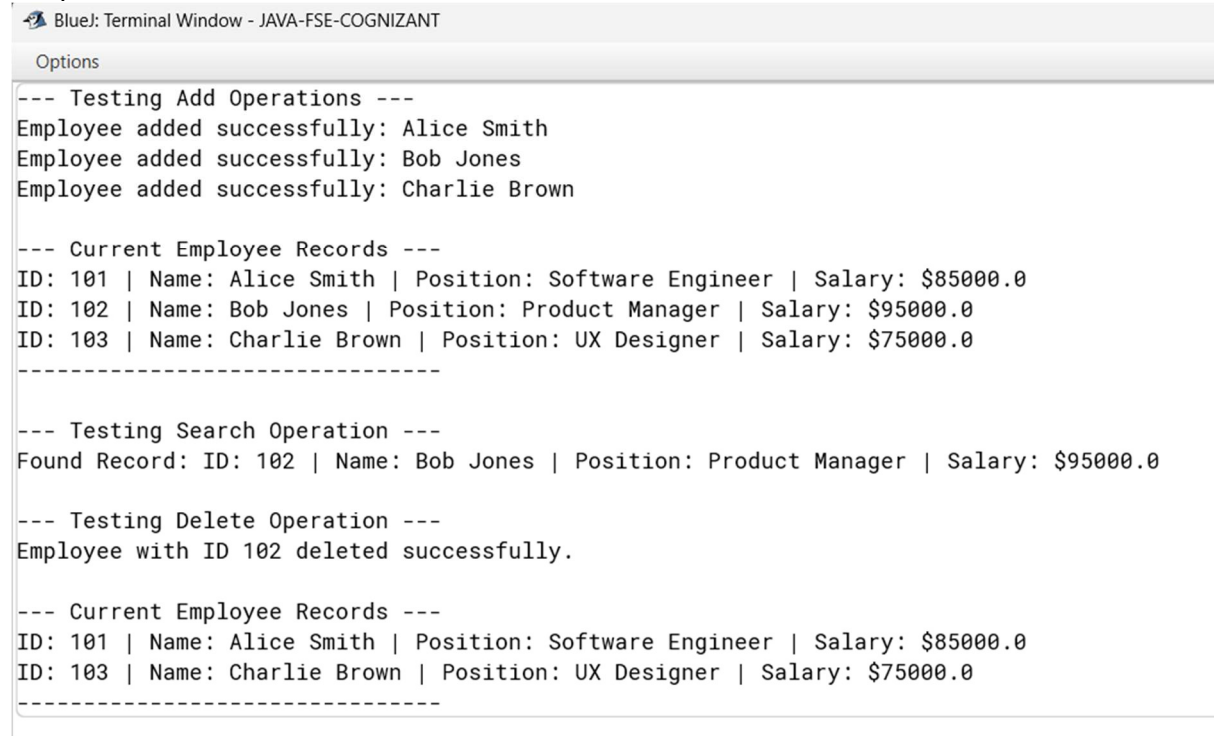
    }

    System.out.println("\n--- Testing Delete Operation ---");
    manager.deleteEmployee(102);

    manager.traverseEmployees();
}
}

```

Output:



```

BlueJ: Terminal Window - JAVA-FSE-COGNIZANT

Options

--- Testing Add Operations ---
Employee added successfully: Alice Smith
Employee added successfully: Bob Jones
Employee added successfully: Charlie Brown

--- Current Employee Records ---
ID: 101 | Name: Alice Smith | Position: Software Engineer | Salary: $85000.0
ID: 102 | Name: Bob Jones | Position: Product Manager | Salary: $95000.0
ID: 103 | Name: Charlie Brown | Position: UX Designer | Salary: $75000.0
-----

--- Testing Search Operation ---
Found Record: ID: 102 | Name: Bob Jones | Position: Product Manager | Salary: $95000.0

--- Testing Delete Operation ---
Employee with ID 102 deleted successfully.

--- Current Employee Records ---
ID: 101 | Name: Alice Smith | Position: Software Engineer | Salary: $85000.0
ID: 103 | Name: Charlie Brown | Position: UX Designer | Salary: $75000.0
-----

```

Question – 5

2,3.

Code:

```

class Task {
    int taskId;
    String taskName;
    String status;
    Task next;

    public Task(int taskId, String taskName, String status) {
        this.taskId = taskId;
        this.taskName = taskName;
        this.status = status;
        this.next = null;
    }
}

class TaskLinkedList {
    private Task head = null;

    public void addTask(int taskId, String taskName, String status) {
        Task newTask = new Task(taskId, taskName, status);
        if (head == null) {
            head = newTask;
            return;
        }
        Task current = head;
    }
}

```



```

        while (current.next != null) {
            current = current.next;
        }
        current.next = newTask;
    }

    public Task searchTask(int taskId) {
        Task current = head;
        while (current != null) {
            if (current.taskId == taskId) {
                return current;
            }
            current = current.next;
        }
        return null;
    }

    public boolean deleteTask(int taskId) {
        if (head == null) {
            return false;
        }
        if (head.taskId == taskId) {
            head = head.next;
            return true;
        }
        Task current = head;
        while (current.next != null && current.next.taskId != taskId) {
            current = current.next;
        }
        if (current.next != null) {
            current.next = current.next.next;
            return true;
        }
        return false;
    }

    public void traverseTasks() {
        Task current = head;
        if (current == null) {
            System.out.println("No tasks found.");
            return;
        }
        while (current != null) {
            System.out.println("ID: " + current.taskId + ", Name: " + current.taskName + ", Status: " + current.status);
            current = current.next;
        }
    }

    public static void main(String[] args) {
        TaskLinkedList list = new TaskLinkedList();

        list.addTask(101, "Design Database", "In Progress");
        list.addTask(102, "Develop API", "Pending");
        list.addTask(103, "Write Unit Tests", "Pending");
    }

```

```

        System.out.println("Initial Task List:");
        list.traverseTasks();
        System.out.println();

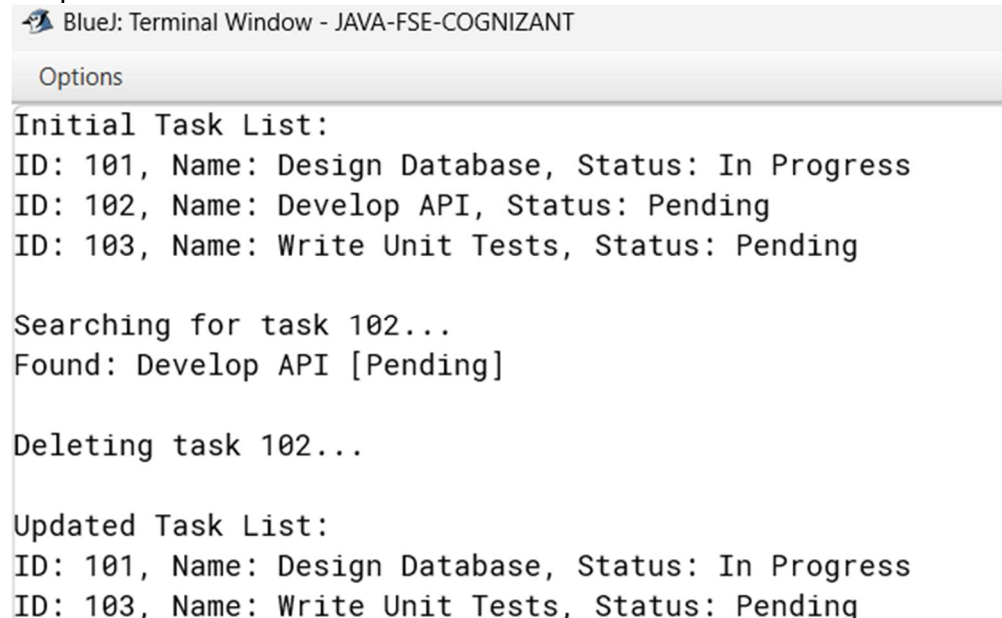
        System.out.println("Searching for task 102...");
        Task found = list.searchTask(102);
        if (found != null) {
            System.out.println("Found: " + found.taskName + " [" + found.status + "]");
        } else {
            System.out.println("Task not found.");
        }
        System.out.println();

        System.out.println("Deleting task 102...");
        list.deleteTask(102);
        System.out.println();

        System.out.println("Updated Task List:");
        list.traverseTasks();
    }
}

```

Output:



```

Blue: Terminal Window - JAVA-FSE-COGNIZANT

Options

Initial Task List:
ID: 101, Name: Design Database, Status: In Progress
ID: 102, Name: Develop API, Status: Pending
ID: 103, Name: Write Unit Tests, Status: Pending

Searching for task 102...
Found: Develop API [Pending]

Deleting task 102...

Updated Task List:
ID: 101, Name: Design Database, Status: In Progress
ID: 103, Name: Write Unit Tests, Status: Pending

```

Question – 6

2,3.

Code:

```

import java.util.Arrays;

class Book {
    int bookId;
    String title;
    String author;

    public Book(int bookId, String title, String author) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
    }
}

```

```

@Override
public String toString() {
    return "Book ID: " + bookId + ", Title: " + title + ", Author: " + author;
}
}

public class BookSearch {
    public static Book linearSearch(Book[] books, String title) {
        for (Book b : books) {
            if (b.title.equalsIgnoreCase(title)) return b;
        }
        return null;
    }

    public static Book binarySearch(Book[] books, String title) {
        int left = 0, right = books.length - 1;
        while (left <= right) {
            int mid = (left + right) / 2;
            int cmp = books[mid].title.compareToIgnoreCase(title);
            if (cmp == 0) return books[mid];
            if (cmp < 0) left = mid + 1;
            else right = mid - 1;
        }
        return null;
    }

    public static void main(String[] args) {
        Book[] books = {
            new Book(1, "Java Basics", "Author A"),
            new Book(2, "Data Structures", "Author B"),
            new Book(3, "Algorithms", "Author C"),
            new Book(4, "Machine Learning", "Author D"),
            new Book(5, "Artificial Intelligence", "Author E")
        };

        Arrays.sort(books, (a, b) -> a.title.compareToIgnoreCase(b.title));

        String t1 = "Machine Learning";
        Book r1 = linearSearch(books, t1);
        System.out.println("Linear Search Result: " + (r1 != null ? r1 : "Not Found"));

        String t2 = "Algorithms";
        Book r2 = binarySearch(books, t2);
        System.out.println("Binary Search Result: " + (r2 != null ? r2 : "Not Found"));

        String t3 = "Python";
        Book r3 = binarySearch(books, t3);
        System.out.println("Binary Search Result: " + (r3 != null ? r3 : "Not Found"));
    }
}

```

Output:

Output

Clear

```

Linear Search Result: Book ID: 4, Title: Machine Learning, Author: Author D
Binary Search Result: Book ID: 3, Title: Algorithms, Author: Author C
Binary Search Result: Not Found

=== Code Execution Successful ===

```

Question – 7

2,3.

Code:

```
class FutureValue {
    public static double calculateFutureValue(double presentValue, double growthRate, int
years) {
        if (years == 0) return presentValue;
        return calculateFutureValue(presentValue * (1 + growthRate), growthRate, years - 1);
    }

    public static void main(String[] args) {
        double presentValue = 1000;
        double growthRate = 0.05;
        int years = 5;
        double futureValue = calculateFutureValue(presentValue, growthRate, years);
        System.out.println("Future Value after " + years + " years: " + futureValue);
    }
}
```

Output:

```
Output

Future Value after 5 years: 1276.2815625000003

=== Code Execution Successful ===
```

Question 1

Explain why data structures and algorithms are essential in handling large inventories. Discuss the types of data structures suitable for this problem.

Large warehouses may contain thousands of products. Efficient data structures help:

1. Store data systematically.
2. Retrieve product information quickly.
3. Update stock efficiently.
4. Reduce memory usage and execution time.

Without proper data structures, operations become slow as inventory grows.

Suitable Data Structures

Data Structure	Advantages	Complexity
ArrayList	Easy to use, dynamic size	Search O(n)
HashMap	Fast lookup using productId	Search O(1) average
TreeMap	Sorted storage	O(log n)

Best Choice: HashMap<Integer, Product> because productId is unique.

Analyze the time complexity of each operation (add, update, delete) in your chosen data structure.

Discuss how you can optimize these operations.

Operation	Time Complexity
Add	O(1)
Update	O(1)
Delete	O(1)
Search	O(1)

Optimization

Use HashMap instead of ArrayList.

Use indexing based on productId.
Use database indexing for persistent storage.

Question 2

Explain Big O notation and how it helps in analyzing algorithms.
Describe the best, average, and worst-case scenarios for search operations.

Big O notation measures how an algorithm's running time grows as input size increases.
Examples:
O(1) → Constant
O(log n) → Logarithmic
O(n) → Linear
O(n²) → Quadratic
Search Cases

Case	Linear Search	Binary Search
Best	O(1)	O(1)
Average	O(n)	O(log n)
Worst	O(n)	O(log n)

Compare the time complexity of linear and binary search algorithms.
Discuss which algorithm is more suitable for your platform and why

Algorithm	Time Complexity
Linear Search	O(n)
Binary Search	O(log n)

Suitable Choice -
Binary Search is better because:
Much faster for large datasets.
Reduces search space by half each step.
Ideal for sorted product catalogs.

Question 3

Explain different sorting algorithms (Bubble Sort, Insertion Sort, Quick Sort, Merge Sort).
Sorting algorithms are fundamental techniques used to arrange data in a particular order, such as ascending or descending. Bubble Sort is a simple algorithm that repeatedly compares adjacent elements and swaps them if they are out of order, making it easy to understand but inefficient for large datasets due to its O(n²) time complexity. Insertion Sort works by building a sorted list one element at a time, inserting each new element into its correct position, which makes it efficient for small or nearly sorted datasets but still has a worst-case complexity of O(n²). Quick Sort, on the other hand, is a divide-and-conquer algorithm that selects a pivot element, partitions the array into two subarrays based on the pivot, and recursively sorts them, achieving an average time complexity of O(nlogn) and making it one of the fastest sorting methods in practice. Merge Sort also uses divide-and-conquer by splitting the array into halves, sorting each half, and merging them back together, ensuring a stable sort with a consistent O(nlogn) complexity. Each algorithm has its strengths and weaknesses, but Quick Sort and Merge Sort are generally preferred for handling large datasets efficiently.

Compare the performance (time complexity) of Bubble Sort and Quick Sort.
Discuss why Quick Sort is generally preferred over Bubble Sort.

Bubble Sort has a time complexity of O(n²) in both average and worst cases, since it repeatedly compares and swaps adjacent elements until the list is sorted, making it inefficient for large datasets. Quick Sort, on the other hand, has an average time complexity of O(nlogn), though its worst case is O(n²) if poor pivot choices are made; however, with good pivot selection strategies (like randomized or median-of-three), Quick Sort usually performs much faster. Quick Sort is generally preferred over Bubble Sort because it is significantly more efficient, scales well with larger inputs, and is widely used in practice due to its divide-and-conquer approach that reduces the number of comparisons and swaps compared to Bubble Sort's repetitive passes.

Question 4

Explain how arrays are represented in memory and their advantages.

Arrays are represented in memory as a contiguous block of locations, where each element is stored sequentially and can be directly accessed using its index. This representation allows for efficient random access, since the address of any element can be calculated quickly using the base address and the index offset. The main advantages of arrays include constant-time access to elements, simplicity in implementation, and ease of iterating through data. Additionally, arrays are memory-efficient compared to linked structures because they avoid the overhead of storing extra pointers, making them ideal for scenarios where fast access and predictable memory usage are important.

Analyze the time complexity of each operation (add, search, traverse, delete).
Discuss the limitations of arrays and when to use them.

In arrays, the time complexity of operations varies: adding an element at the end takes $O(1)$ if space is available, but inserting at a specific position requires shifting elements, making it $O(n)$. Searching is $O(n)$ for linear search, but $O(\log n)$ if the array is sorted and binary search is used. Traversing all elements is always $O(n)$, since each element must be visited. Deleting an element also requires shifting subsequent elements, so it is $O(n)$. Despite these advantages of direct access and predictable memory layout, arrays have limitations: their size is fixed once declared, making dynamic resizing difficult; insertion and deletion are costly due to shifting; and they may waste memory if allocated larger than needed. Arrays are best used when the dataset size is known in advance, random access is frequently required, and operations like traversal or searching dominate over insertion and deletion.

Question 5

Explain the different types of linked lists (Singly Linked List, Doubly Linked List).

A Singly Linked List (SLL) is a linear data structure where each node contains data and a pointer to the next node, forming a one-way chain. It allows efficient insertion and deletion at any position without shifting elements, but traversal is only possible in one direction, making reverse operations costly. In contrast, a Doubly Linked List (DLL) has nodes with two pointers: one pointing to the next node and another to the previous node. This bidirectional structure enables traversal in both directions and simplifies operations like deletion or insertion at both ends, though it requires extra memory for the additional pointer. Overall, SLLs are simpler and more memory-efficient, while DLLs provide greater flexibility and faster operations in scenarios where frequent forward and backward traversal is needed.

Analyze the time complexity of each operation.

Discuss the advantages of linked lists over arrays for dynamic data.

In linked lists, the time complexity of operations depends on the type of list and the position of the element. For a Singly Linked List (SLL), adding at the head is $O(1)$, but inserting at the tail or a specific position requires traversal, making it $O(n)$. Searching is always $O(n)$ since elements must be checked sequentially. Traversal is also $O(n)$, and deletion is $O(1)$ if the node reference is known, but $O(n)$ if the node must first be located. In a Doubly Linked List (DLL), insertion and deletion can be more efficient at both ends because of the extra pointer to the previous node, though searching and traversal remain $O(n)$.

The key advantages of linked lists over arrays for dynamic data are flexibility and efficient memory usage. Unlike arrays, linked lists do not require a fixed size at creation; they can grow or shrink dynamically without costly resizing. Insertions and deletions are faster since they only involve pointer adjustments rather than shifting elements, making linked lists ideal for applications where frequent modifications occur. However, they sacrifice constant-time random access, so they are best used when dynamic resizing and efficient insert/delete operations are more important than fast indexing.

Question 6

Explain linear search and binary search algorithms.

Linear Search is a simple algorithm where each element in the array or list is checked sequentially until the target element is found or the end of the structure is reached. Its time complexity is $O(n)$, since in the worst case every element must be examined. Binary Search, on the other hand, works only on sorted arrays or lists. It repeatedly divides the search interval in half by comparing the target with the middle element, eliminating half of the remaining elements each step. This makes its time complexity $O(\log n)$, which is far more efficient than linear search for large datasets. In summary, linear search is

straightforward and works on unsorted data, while binary search is faster but requires the data to be sorted beforehand.

Compare the time complexity of linear and binary search.

Discuss when to use each algorithm based on the data set size and order.

Linear search is best suited for small or completely unsorted datasets because it examines every element sequentially, making its worst-case time complexity $O(n)$. It requires no structural preparation, meaning you don't have to waste time sorting the data if you only need to run a single lookup. On the other hand, binary search is vastly superior for large, frequently searched datasets, but it strictly requires the data to be sorted beforehand. By repeatedly dividing the search space in half, binary search achieves a highly efficient logarithmic worst-case time complexity of $O(\log n)$, allowing it to scan millions of records in just a handful of operations. Ultimately, choose linear search for small, quick, or chaotic lists, and choose binary search for massive, ordered datasets where lookup speed is critical.

Question 7

Explain the concept of recursion and how it can simplify certain problems.

At its core, recursion is a programming technique where a function solves a problem by breaking it down into smaller instances of itself. Every recursive function relies on two essential pieces to avoid running forever: a base case, which acts as the exit condition that stops the function from calling itself, and a recursive step, which moves the problem closer to that base case with each step. This approach simplifies complex problems by allowing you to focus on solving a single, manageable piece of the puzzle at a time rather than managing the intricate loops and states required by an iterative solution.

Recursion shines when dealing with problems that have a naturally repetitive or nested structure. For instance, tasks like navigating a computer's file directory, parsing nested data formats, or working with complex data structures like trees and graphs are inherently recursive because a single folder can contain other folders, which in turn contain more folders. Instead of writing massive loops to keep track of every level, a recursive function cleanly handles the current level and hands off the sub-levels to a fresh copy of itself, resulting in elegant, readable, and shorter code.

Discuss the time complexity of your recursive algorithm.

Explain how to optimize the recursive solution to avoid excessive computation.

The recursive algorithm for calculating future value has a time complexity of $O(n)$ because it makes one recursive call for each year until the base case is reached, and it also requires $O(n)$ space due to the call stack. While this approach is simple and clear, it can become inefficient for large values of n since deep recursion may lead to stack overflow and unnecessary repeated computation. To optimize, the best solution is to use the direct mathematical formula $FV = PV \cdot (1+r)^n$, which computes the result in $O(1)$ time and space. Another optimization technique is memoization, where previously computed results are stored and reused to avoid redundant calculations, especially useful when multiple queries overlap. In practice, the direct formula is the most efficient and avoids the overhead of recursion entirely.